



LangChain Agents

Basics

For complex ideas involving schema, memory, invoking etc refer to:

 [LangGraph](#)

`create_agent`

- Creates a simple ReACT agent with tools node
- For anything more complex than ReACT we should use LangGraph

```
from langchain.agents import create_agent
```

```
def get_weather(city: str) → str:  
    """Get weather for a given city."""  
    return f"It's always sunny in {city}!"
```

```
agent = create_agent(  
    # LangChain chat model  
    model=model,  
    system_prompt=SYSTEM_PROMPT,  
    tools=[get_user_location, get_weather_for_location],  
    context_schema=Context,  
    response_format=ToolStrategy(ResponseFormat),  
    checkpoint=checkpoint  
)
```

```
config = {"configurable": {"thread_id": "1"}}
```

```
# Run the agent  
response = agent.invoke(  
    {"messages": [{"role": "user", "content": "what is the weather outside?"}]},  
    config=config,  
    context=Context(user_id="1")  
)
```

▼ Customise State Demo

- Build on top of `AgentState`
- Note: for `create_agent` only TypedDict is supported

```

from langchain.agents import AgentState

class CustomState(AgentState):
    user_preferences: dict

agent = create_agent(
    model,
    tools=[tool1, tool2],
    state_schema=CustomState
)
# The agent can now track additional state beyond messages
result = agent.invoke({
    "messages": [{"role": "user", "content": "I prefer technical explanations"}],
    "user_preferences": {"style": "technical", "verbosity": "detailed"},
})

```

▼ Streaming Demo

```

for chunk in agent.stream({
    "messages": [{"role": "user", "content": "Search for AI news and summarize the findings"}],
    stream_mode="values"):
    # Each chunk contains the full state at that point
    latest_message = chunk["messages"][-1]
    if latest_message.content:
        print(f"Agent: {latest_message.content}")
    elif latest_message.tool_calls:
        print(f"Calling tools: {[tc['name'] for tc in latest_message.tool_calls]}")

```

▼ Dynamic Model Demo

- Allows us to select which model to use based on conditions
- To use a dynamic model, create middleware using the `@wrap_model_call` decorator that modifies the model in the request:

```
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from langchain.agents.middleware import wrap_model_call, ModelRequest, ModelResponse

basic_model = ChatOpenAI(model="gpt-4o-mini")
advanced_model = ChatOpenAI(model="gpt-4o")

@wrap_model_call
def dynamic_model_selection(request: ModelRequest, handler) → ModelResponse:
    """Choose model based on conversation complexity."""
    message_count = len(request.state["messages"])

    if message_count > 10:
        # Use an advanced model for longer conversations
        model = advanced_model
    else:
        model = basic_model

    return handler(request.override(model=model))

agent = create_agent(
    model=basic_model, # Default model
    tools=tools,
    middleware=[dynamic_model_selection]
)
```

▼ Structured Output from Agent Demo

Option 1 (General)

- Lets us run an agent to do its normal thing but then we say the final response back to ius must be a structured output:

```
from pydantic import BaseModel
from langchain.agents import create_agent
from langchain.agents.structured_output import ToolStrategy

class ContactInfo(BaseModel):
    name: str
    email: str
    phone: str

agent = create_agent(
    model="gpt-4o-mini",
    tools=[search_tool],
    response_format=ToolStrategy(ContactInfo)
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Extract contact info from: John Doe, john@example.com, (555) 123-4567"}]
})

# To access the final response we run:
result["structured_response"]
# ContactInfo(name='John Doe', email='john@example.com', phone='(555) 123-4567')
```

Option 2 (Best if Possible)

- `ProviderStrategy` uses the model provider's native structured output generation (but only works for some providers which have this feature such as OpenAI)

```
from langchain.agents.structured_output import ProviderS
trategy

agent = create_agent(
    model="gpt-4o",
    response_format=ProviderStrategy(ContactInfo)
)
```

▼ Dynamic System Prompt Demo

- `@dynamic_prompt` middleware lets us create changing system prompts based on conditions

```
from typing import TypedDict
from langchain.agents import create_agent
from langchain.agents.middleware import dynamic_prompt, ModelRequest

class Context(TypedDict):
    user_role: str

@dynamic_prompt
def user_role_prompt(request: ModelRequest) → str:
    """Generate system prompt based on user role."""
    user_role = request.runtime.context.get("user_role", "user")
    base_prompt = "You are a helpful assistant."

    if user_role == "expert":
        return f"{base_prompt} Provide detailed technical responses."
    elif user_role == "beginner":
        return f"{base_prompt} Explain concepts simply and avoid jargon."

    return base_prompt

agent = create_agent(
    model="gpt-4o",
    tools=[web_search],
    middleware=[user_role_prompt],
    context_schema=Context
)
```



```
# The system prompt will be set dynamically based on context
result = agent.invoke(
    {"messages": [{"role": "user", "content": "Explain machine learning"}]},
    context={"user_role": "expert"}
)
```